Performance & CI/CD Workflow

1. Does using YAML instead of properties affect performance or startup time? Why or why not?

A: No, using **YAML** instead of properties files **does not significantly affect performance or startup time**. This is because Spring Boot converts both formats into the same internal **Environment** object very early during startup. The time difference between parsing the two formats is negligible compared to the time spent on component scanning or bean creation.

2. Which CI/CD tools do you use for building, testing, and deploying your application, and how do they fit into your workflow?

A: I typically use **Jenkins** or **GitLab CI** for the main pipeline orchestration. They fit into the workflow by triggering automatically when code is pushed to Git. They handle three stages: **1) Build** (running mvn clean install), **2) Test** (running unit and integration tests), and **3) Deploy** (building the Docker image and pushing it to the target environment like Kubernetes).

3. What steps do you take to diagnose a pipeline that fails randomly?

A: I take three main steps:

1. **Check for External Flakiness:** Review logs to see if external dependencies (like a database or an external API) were unavailable or timed out, causing random failures.
2. **Verify Tests:** Check if any **unit or integration tests** are not truly isolated and rely on specific system timing, which is a common source of flakiness.
3. **Resource Check:** Ensure the CI/CD agent is not running out of resources (CPU or memory) during the peak load of the testing phase.

4. How do Docker containers improve the deployment process in microservices?

A: Docker containers package the application and all its required settings and libraries into one clean, portable unit. This improves deployment by guaranteeing that the service runs the **exact same way** on a developer's laptop as it does in production. This eliminates "it works on my machine" problems and simplifies rolling updates.

JAVA Interview Buddy

5. Why are health checks important, and how do you configure them in Spring Boot?

A: Health checks are important because they allow deployment systems (like Kubernetes) to monitor the application's actual operational status, not just if the server is running. You configure them using **Spring Boot Actuator**; the built-in /actuator/health endpoint automatically checks the database connection, disk space, and other vital components.

Deployment & Troubleshooting

6. How do you manage different configurations for dev, QA, UAT, and prod during deployments?

A: I manage configurations using **Spring Profiles** (e.g., application-dev.yml). I store environment-specific values in external sources like a **Configuration Server** or **Kubernetes ConfigMaps**. During deployment, I simply set the SPRING_PROFILES_ACTIVE environment variable (e.g., prod) so the application loads the correct settings for that environment.

7. What should you verify before deploying an application to the cloud for the first time?

A: I verify three critical things:

1. **External Secrets:** Ensure database passwords and API keys are read from a secure **Secret Manager** (like Vault), not from committed configuration files.
2. **Health Checks:** Verify the /actuator/health endpoint is correct and accurate.
3. **Logging:** Ensure structured logging is configured and flowing correctly to the centralized logging system (ELK/Loki).

8. What do you do when a Jenkins agent runs out of disk space during builds?

A: The immediate fix is to **clean the workspace** and delete old, cached Maven dependencies by running commands like mvn clean and clearing the local Maven repository (~/.m2/repository). For a long- term solution, I would adjust the agent's monitoring to proactively alert when disk space reaches 80% capacity and increase the storage volume size.

JAVA Interview Buddy

9. How do you reduce the size of a large Docker image to speed up deployments?

A: I reduce the image size by using **multi-stage builds** in the Dockerfile. The first stage builds the application using a large SDK image, and the second, final stage copies only the minimal executable **JAR file** and a lightweight Java Runtime Environment (like Alpine or a minimal JRE base image). This drastically cuts the size and download time.

JAVA Interview Buddy